Experiment 23: The prolog program to implement a family tree.

Aim: To write and implement a Prolog program to represent a family tree and identify relationships such as parent, father, mother, sibling, grandparent, and ancestor.

Algorithm:

1. Start the program.
2. Define family relationships using facts such as parent, male, and female.
3. Define rules to identify father and mother based on parent relationships and gender.
4. Define the grandparent rule by checking whether a person is the parent of another person's parent.
5. Define the sibling rule by checking whether two different persons share a parent.
6. Define the ancestor rule using recursion to identify direct and indirect ancestors.
7. Load the program in SWI-Prolog.
8. Execute queries to verify the family relationships.
9. Display the results and stop.

Code:

family_tree.pl

```
% Facts: Parent relationships
parent(john, peter).
parent(john, mary).
parent(peter, david).
parent(peter, lisa).
parent(mary, sam).
parent(mary, rita).

% Rules
% X is a father of Y
father(X, Y) :-
    parent(X, Y),
    male(X).

% X is a mother of Y
mother(X, Y) :-
    parent(X, Y),
    female(X).

% X is a grandparent of Y
grandparent(X, Y) :-
    parent(X, Z),
    parent(Z, Y).

% X is a sibling of Y
sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.

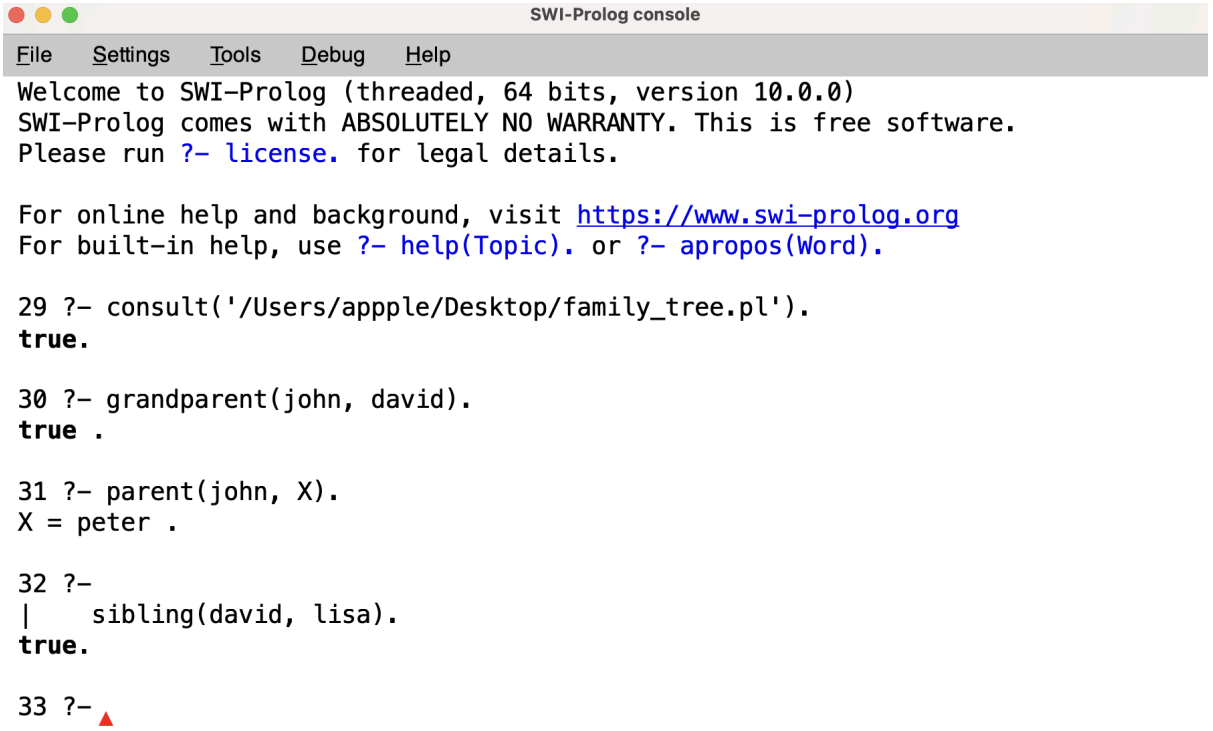
% X is an ancestor of Y
ancestor(X, Y) :-
    parent(X, Y).

ancestor(X, Y) :-
    parent(X, Z),
    ancestor(Z, Y).

% Gender facts
male(john).
male(peter).
male(david).
male(sam).

female(mary).
female(lisa).
female(rita).
```

Output:



Result: Thus, the Prolog program to implement a family tree was successfully written and executed. It identifies family relationships such as parents, father, mother, siblings, grandparents, and ancestors using facts and rules.

Experiment 24: Prolog Program to suggest Dieting System based on Disease.

Aim: To write and implement a Prolog program that suggests a suitable diet based on the disease specified by the user.

Algorithm:

1. Start the program.
2. Define diseases and their corresponding diet recommendations as facts.
3. Create a rule to retrieve a diet recommendation based on the given disease.
4. Accept a disease name as input through a query.
5. Check whether a diet recommendation exists for the disease.
6. If a recommendation exists, display the disease and suggested diet.
7. Otherwise, display a message indicating that no recommendation is available.
8. Stop the program.

Code:

diet_system.pl

family_tree.pl | diet_system.pl

```
% Diet Suggestion System Based on Disease

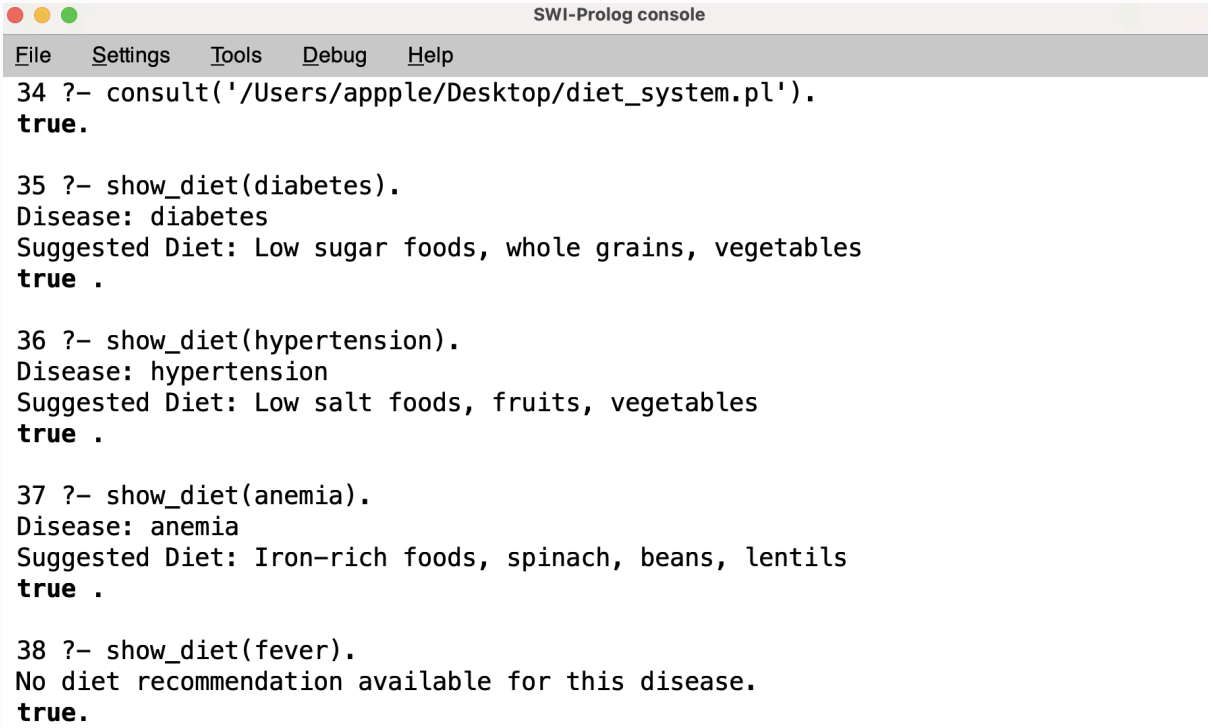
% Diet recommendations for diseases
diet(diabetes, 'Low sugar foods, whole grains, vegetables').
diet(hypertension, 'Low salt foods, fruits, vegetables').
diet(obesity, 'Balanced diet, vegetables, fruits, lean protein').
diet(anemia, 'Iron-rich foods, spinach, beans, lentils').
diet(heart_disease, 'Low saturated fat foods, fruits, vegetables').
diet(acidity, 'Small meals, mild foods, avoid spicy foods').

% Rule to suggest diet
suggest_diet(Disease, Diet) :-
    diet(Disease, Diet).

% Display diet recommendation
show_diet(Disease) :-
    suggest_diet(Disease, Diet),
    write('Disease: '),
    write(Disease),
    nl,
    write('Suggested Diet: '),
    write(Diet),
    nl.

% Handle unknown diseases
show_diet(Disease) :-
    \+ diet(Disease, _),
    write('No diet recommendation available for this disease.'),
    nl.
```

Output:



Result: Thus, the Prolog program for a diet suggestion system based on disease was successfully implemented. It provides diet suggestions for predefined diseases and displays a message for unknown diseases.

Experiment 25:Prolog program to implement Monkey Banana Problem

Aim: To write and implement a Prolog program to solve the Monkey Banana Problem using state-space representation and logical rules.

Algorithm:

1. Start the program.
2. Define the initial state of the monkey, box, and bananas.
3. Define the possible actions: walk, push the box, climb the box, and grasp the bananas.
4. Represent each action as a transition from one state to another.
5. Define the goal state in which the monkey has the bananas.
6. Search for a sequence of actions from the initial state to the goal state.
7. Display the solution steps.
8. Stop the program.

Code:

monkey_banana.pl

family_tree.pl | diet_system.pl | monkey_banana.pl

```
% Monkey Banana Problem

% Initial state
initial(state(monkey_at(door),
              box_at(window),
              bananas_at(middle),
              on_floor)).

% Monkey walks to the box
move(state(monkey_at(door), box_at(window),
           bananas_at(middle), on_floor),
     walk(door, window),
     state(monkey_at(window), box_at(window),
           bananas_at(middle), on_floor)).

% Monkey pushes the box under the bananas
move(state(monkey_at(window), box_at(window),
           bananas_at(middle), on_floor),
     push_box(window, middle),
     state(monkey_at(middle), box_at(middle),
           bananas_at(middle), on_floor)).

% Monkey climbs the box
move(state(monkey_at(middle), box_at(middle),
           bananas_at(middle), on_floor),
     climb,
     state(monkey_at(middle), box_at(middle),
           bananas_at(middle), on_box)).

% Monkey grasps the bananas
move(state(monkey_at(middle), box_at(middle),
           bananas_at(middle), on_box),
     grasp,
     state(monkey_at(middle), box_at(middle),
           bananas_at(middle), has_bananas)).

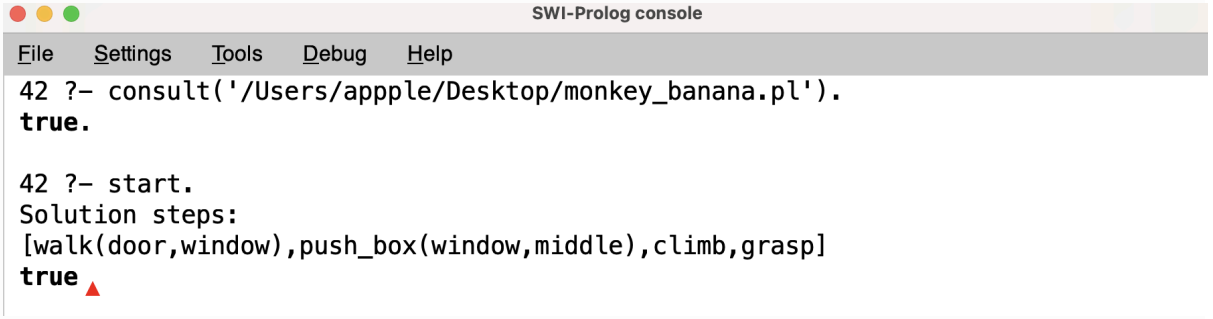
% Goal state
goal(state(_, _, _, has_bananas)).

% Solve the problem
solve(State, []) :-
    goal(State).

solve(State, [Action | Rest]) :-
    move(State, Action, NextState),
    solve(NextState, Rest).

% Display solution
start :-
    initial(State),
    solve(State, Actions),
    write('Solution steps:'), nl,
    write(Actions), nl.
```

Output:



Result: Thus, the Prolog program for the Monkey Banana Problem was implemented using state-space representation and logical rules. The program finds a sequence of actions that allows the monkey to obtain the bananas.

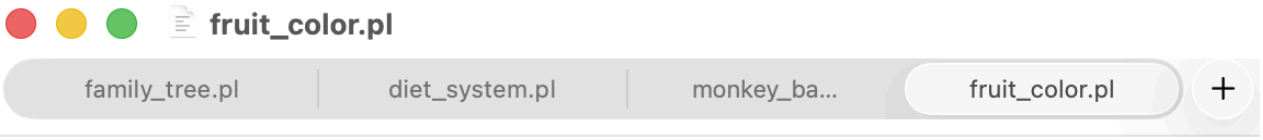
Experiment 26: Prolog Program for fruit and its color using Back Tracking.

Aim: To write and implement a Prolog program to identify fruits and their colors using backtracking.

Algorithm:

1. Start the program.
2. Define facts representing fruits and their corresponding colors.
3. Define a rule to find the color of a given fruit.
4. Enter a query to find a fruit's color or list fruits of a particular color.
5. Prolog searches the facts and returns a matching result.
6. Press the semicolon key to backtrack and find the next matching result.
7. Stop when no more matching results are available.

Code:

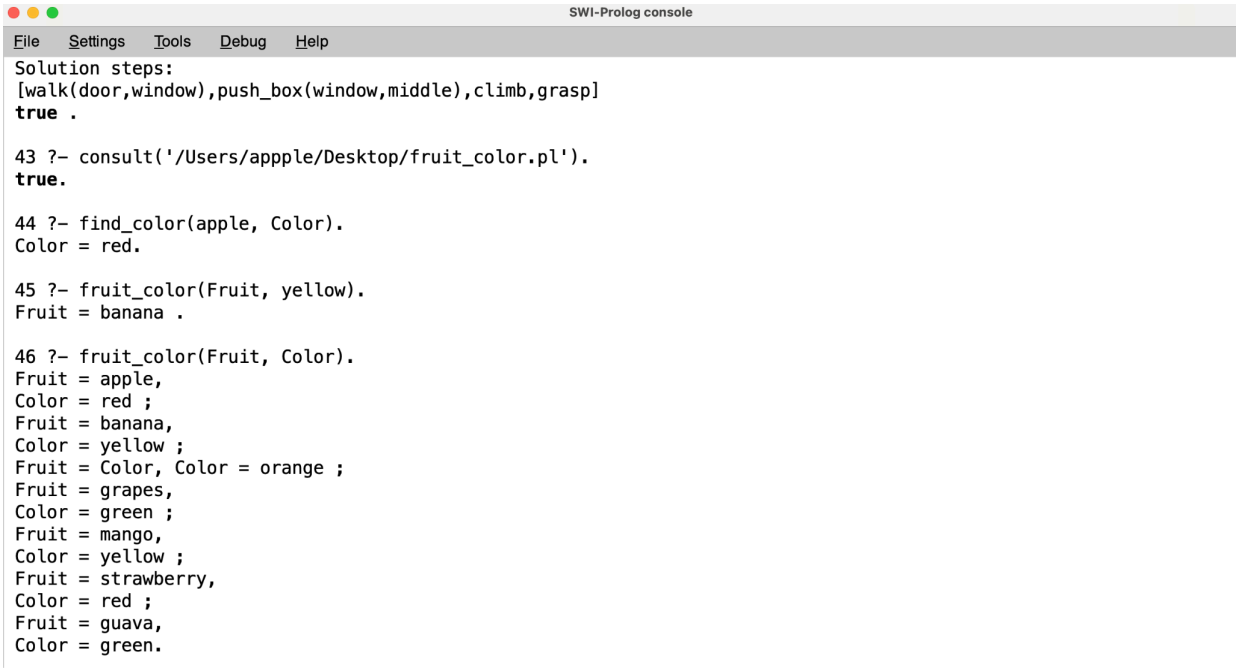



```
% Facts: Fruits and their colors

fruit_color(apple, red).
fruit_color(banana, yellow).
fruit_color(orange, orange).
fruit_color(grapes, green).
fruit_color(mango, yellow).
fruit_color(strawberry, red).
fruit_color(guava, green).

% Rule to find the color of a fruit
find_color(Fruit, Color) :-
    fruit_color(Fruit, Color).
```

Output:



Result: Thus, the Prolog program to identify fruits and their colors using backtracking was successfully implemented. Prolog returns matching solutions one by one until no further solutions are available.

Experiment 27: Prolog Program to implement Best First Search algorithm

Aim: To write and implement a Prolog program to perform the Best First Search algorithm using heuristic values to find a path from the start node to the goal node.

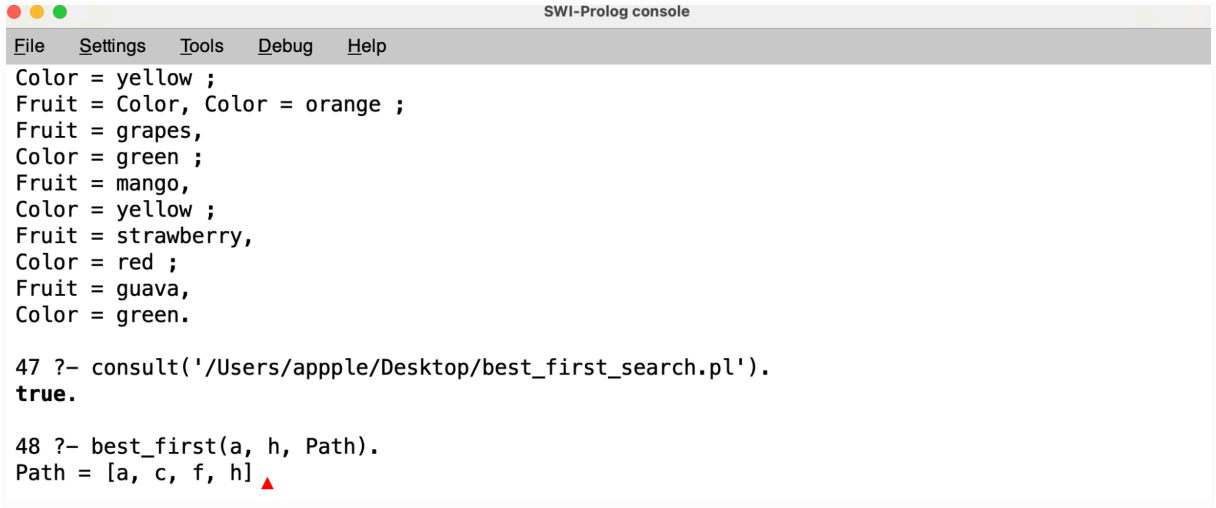
Algorithm:

1. Start the program.
2. Define the graph using edges between nodes.
3. Assign a heuristic value to each node, indicating its estimated distance from the goal.
4. Place the start node in the search list.
5. Select the node with the lowest heuristic value.
6. Check whether the selected node is the goal.
7. If it is not the goal, expand its unvisited neighboring nodes.
8. Sort the available paths according to their heuristic values.
9. Repeat until the goal is found.
10. Display the path and stop.

Code:



Output:



Result: Thus, the Prolog program for the Best First Search algorithm was implemented successfully. It uses heuristic values to select the most promising node and find a path from the start node to the goal node.

Experiment 28: The Prologue program for Medical Diagnosis

Aim: To write and implement a Prolog program for medical diagnosis by identifying possible diseases based on the symptoms provided by the user.

Algorithm:

1. Start the program.
2. Define diseases and their associated symptoms as facts.
3. Create a diagnosis rule to match a disease with its symptoms.
4. Accept a symptom as a query.
5. Search the facts to identify diseases associated with the given symptom.
6. Use backtracking to find all matching diseases.
7. Display the possible diseases.
8. Stop the program.

Code:

medical_diagnosis.pl

di | monkey_b... | fruit_color... | best_first... | medical_d... | +

```
% Medical Diagnosis System

% Symptoms of diseases
symptom(flu, fever).
symptom(flu, cough).
symptom(flu, body_pain).

symptom(common_cold, cough).
symptom(common_cold, sneezing).
symptom(common_cold, runny_nose).

symptom(malaria, fever).
symptom(malaria, chills).
symptom(malaria, sweating).

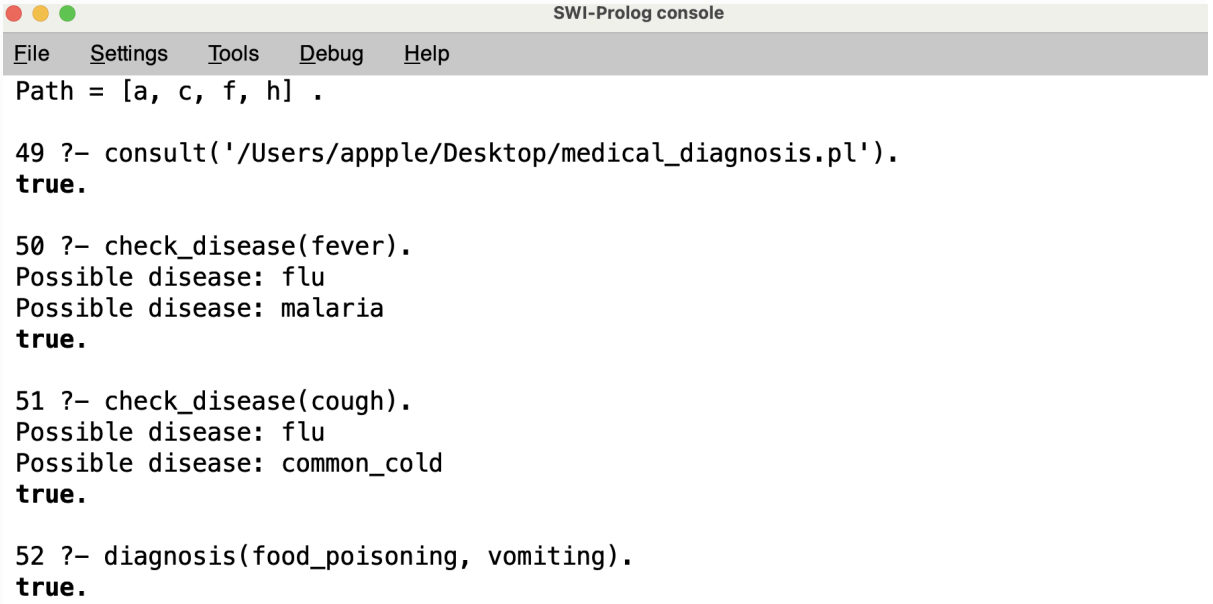
symptom(food_poisoning, vomiting).
symptom(food_poisoning, stomach_pain).
symptom(food_poisoning, diarrhea).

% Diagnosis rule
diagnosis(Disease, Symptom) :-
    symptom(Disease, Symptom).

% Display diagnosis based on symptom
check_disease(Symptom) :-
    diagnosis(Disease, Symptom),
    write('Possible disease: '),
    write(Disease),
    nl,
    fail.

check_disease(_).
```

Output:



Result : Thus, the Prolog program for medical diagnosis was implemented successfully. It identifies possible diseases based on the symptoms provided and uses backtracking to display multiple matches.

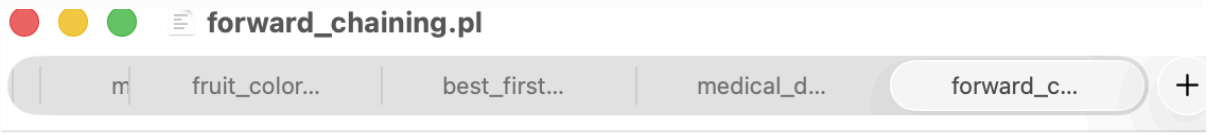
Experiment 29: Prolog Program for forward Chaining. Incorporate required queries.

Aim: To write and implement a Prolog program to demonstrate forward chaining by deriving new facts from existing facts and rules.

Algorithm:

1. Start the program.
2. Define the initial facts.
3. Define rules that derive new facts from existing facts.
4. Check whether the conditions of a rule are satisfied.
5. If the rule produces a new fact, add it to the knowledge base using assertz/1.
6. Repeat the process until no new facts can be derived.
7. Display the derived facts and completion message.
8. Stop the program.

Code:




```
% Forward Chaining Program

% Allow facts to be added dynamically
:- dynamic fact/1.

% Initial facts
fact(sunny).
fact(weekend).

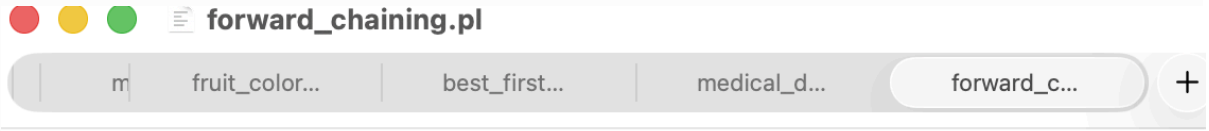
% Rules
rule(go_outside) :-
    fact(sunny),
    fact(weekend).

rule(play_cricket) :-
    fact(go_outside).

% Forward chaining process
forward_chaining :-
    rule(NewFact),
    \+ fact(NewFact),
    !,
    assertz(fact(NewFact)),
    write('New fact derived: '),
    write(NewFact),
    nl,
    forward_chaining.

forward_chaining :-
    write('Forward chaining completed.'),
    nl.
```

Output:




```
% Forward Chaining Program

% Allow facts to be added dynamically
:- dynamic fact/1.

% Initial facts
fact(sunny).
fact(weekend).

% Rules
rule(go_outside) :-
    fact(sunny),
    fact(weekend).

rule(play_cricket) :-
    fact(go_outside).

% Forward chaining process
forward_chaining :-
    rule(NewFact),
    \+ fact(NewFact),
    !,
    assertz(fact(NewFact)),
    write('New fact derived: '),
    write(NewFact),
    nl,
    forward_chaining.

forward_chaining :-
    write('Forward chaining completed.'),
    nl.
```

Result: The error is fixed by declaring fact/1 as a dynamic predicate. The program now derives new facts from existing facts and rules until no further facts can be generated.